JobSheet - Week 7 – Multi-Thread

Nama: Zahra Azkiya Rahmani

NIM: 251524124

Kelas: 1D-D4

Repo GitHub: https://github.com/yayyzea/1D_251524124_Teknik-Pemrograman-PR-.git

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W7_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W7_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Transfer Fulus

Soal Praktikum

Instruksi Pengerjaan:
a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!
b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!
 	
Source Code:
class Account {
  int balance = 150;

```
}

public class TransferFulus {
  public static void main(String[] args) throws InterruptedException {
    Account acc1 = new Account();
    Account acc2 = new Account();

    // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
    Thread t1 = new Thread(() -> {
      synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {} // Simulasi dengan
memberikan jeda. Mengapa diperlukan Exception?

        synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc2.balance += acc1.balance;
        }
      }
    });

    // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
    Thread t2 = new Thread(() -> {
      synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc1.balance += acc2.balance;
        }
      }
    });

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    System.out.println("--- HASIL AKHIR ---");
    System.out.println("Saldo Akhir acc1: " + acc1.balance);
                System.out.println("Saldo Akhir acc2: " + acc2.balance);
```

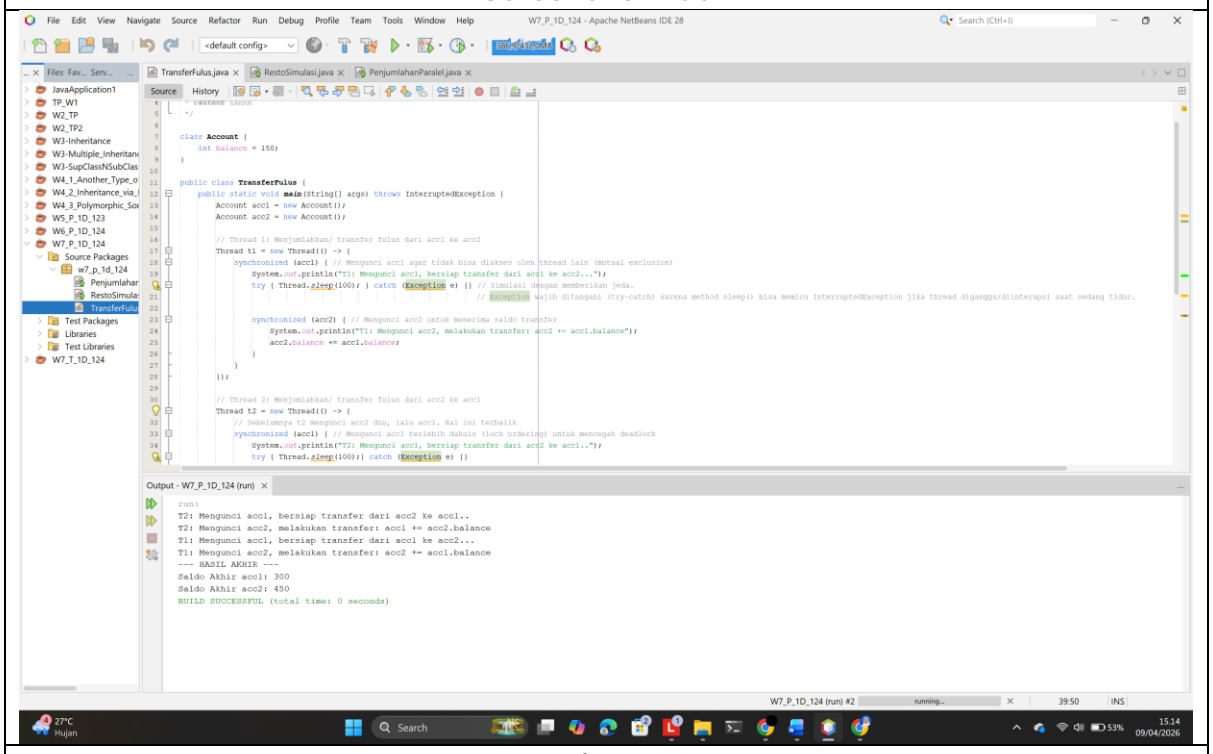
```
    }
}
```

Source Code

```
package w7_p_1d_124;

/**
 * @author ZAHRA
 */

class Account {
   int balance = 150;
}

public class TransferFulus {
   public static void main(String[] args) throws InterruptedException {
      Account acc1 = new Account();
      Account acc2 = new Account();

      // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
      Thread t1 = new Thread(() -> {
         synchronized (acc1) { // Mengunci acc1 agar tidak bisa diakses oleh thread lain (mutual exclusion)
            System.out.println("T1: Mengunci acc1, bersiap transfer dari acc1 ke acc2...");
            try { Thread.sleep(100); } catch (Exception e) {} // Simulasi dengan memberikan jeda.
                                    // Exception wajib ditangani (try-catch) karena method sleep() bisa
memicu InterruptedException jika thread diganggu/diinterupsi saat sedang tidur.

            synchronized (acc2) { // Mengunci acc2 untuk menerima saldo transfer
               System.out.println("T1: Mengunci acc2, melakukan transfer: acc2 += acc1.balance");
               acc2.balance += acc1.balance;
            }
         }
      });

      // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
      Thread t2 = new Thread(() -> {
         // Sebelumnya t2 mengunci acc2 duu, lalu acc1. Hal ini terbalik
         synchronized (acc1) { // Mengunci acc1 terlebih dahulu (lock ordering) untuk mencegah deadlock
            System.out.println("T2: Mengunci acc1, bersiap transfer dari acc2 ke acc1..");
            try { Thread.sleep(100);} catch (Exception e) {}

            synchronized (acc2) { // Mengunci acc2 setelah acc1 berhasil diamankan
               System.out.println("T2: Mengunci acc2, melakukan transfer: acc1 += acc2.balance");
               acc1.balance += acc2.balance;
            }
         }
      });

      t1.start();
      t2.start();

      t1.join();
      t2.join();

      System.out.println("--- HASIL AKHIR ---");
      System.out.println("Saldo Akhir acc1: " + acc1.balance);
                        System.out.println("Saldo Akhir acc2: " + acc2.balance);
```

```
    }
}
```

Screenshot Hasil



Penjelasan

Pada soal ini masalah yang terjadi adalah deadlock, di mana dua thread saling menunggu resource yang dipegang di thread lain, sehingga program mengalami hang/freeze karena tidak satupun yang bisa melanjutkan eksekusi. Hal ini dipicu penguncian silang, di mana Thread1 mengunci acc1 lalu acc2, sedangkan Thread2 mengunci acc2 lalu acc1.

Solusi untuk mengatasi masalah deadlock ini dilakukan dengan menerapkan lock ordering. Urutan penguncian pada Thread2 diselaraskan dengan Thread1.
Thread1 : acc1 dulu, lalu acc2
Thread2 : acc2 dulu, lalu acc1 diselaraskan menjadi acc1 dulu, lalu acc2
Dengan urutan searah ini, circular wait berhasil diputus.

Selain itu, program juga menggunakan synchronized pada object account yang untuk menerapkan prinsip mutual exclusion yang menjamin bahwa hanya ada satu thread yang dapat mengakses dan memodifikasi saldo akun pada satu waktu.

Kemudian, untuk penggunaan Exception pada Thread.sleep(100), hal tersebut digunakan untuk menangani InteruptedException yang diperlukan untuk mengatasi jika thread diinterupsi atau dipaksa berhenti oleh sistem/proses lain saat sedang dalam status jeda (tertidur). Selain itu di bagian akhir kode, digunakan pula method.join() pada looping eksekusi untuk menahan main thread menunggu semua thread menyelesaikan tugasnya sebelum mencetak hasil akhir.

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Resto Ayam

Soal Praktikum

Instruksi Pengerjaan:

a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!

b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!

Source Code:

```
class Resto {
  private int chickenStock = 100;

  public void serveCustomer(String cashierName) {
    if (chickenStock > 0) {
      try { Thread.sleep(10); } catch (InterruptedException e) {}

      chickenStock--;
      System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " +
chickenStock);
    } else {
      System.out.println(cashierName + " gagal: Stok Habis!");
    }
  }

  public int getRemainingStock() {
    return chickenStock;
  }
}

public class RestoSimulasi {
  public static void main(String[] args) throws InterruptedException {
    Resto ayamJuicyLuicyGallagher = new Resto();

    Runnable task = () -> {
      for (int i = 0; i < 40; i++) {

ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
      }
    };

    Thread kasir1 = new Thread(task, "Kasir-A");
```

```
    Thread kasir2 = new Thread(task, "Kasir-B");
    Thread kasir3 = new Thread(task, "Kasir-C");

    kasir1.start();
    kasir2.start();
    kasir3.start();

    kasir1.join();
    kasir2.join();
    kasir3.join();

    System.out.println("--- HASIL AKHIR STOK: " +
ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
  }
}
```

Source Code

- **Synchronized**

```
package w7_p_1d_124;

/**
 * @author ZAHRA
 */

class Resto {
  private int chickenStock = 100;

  public synchronized void serveCustomer(String cashierName) {
  // Method serveCustomer() diakses oleh banyak thread secara bersamaan sehingga terjadi race
condition, akibatnya stok bisa negatif dan tidak konsisten
  // synchronized memastikan hanya satu thread yang dapat mengeksekusi method ini pada satu waktu
(mutual exclusion)
    if (chickenStock > 0) {
      try { Thread.sleep(10); } catch (InterruptedException e) {}
      chickenStock--;
      System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " + chickenStock);
    } else {
      System.out.println(cashierName + " gagal: Stok Habis!");
    }
  }

  public int getRemainingStock() {
    return chickenStock;
  }
}

public class RestoSimulasi {
  public static void main(String[] args) throws InterruptedException {
    Resto ayamJuicyLuicyGallagher = new Resto();

    Runnable task = () -> {
      for (int i = 0; i < 40; i++) {
        ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
      }
```

```
    };

    Thread kasir1 = new Thread(task, "Kasir-A");
    Thread kasir2 = new Thread(task, "Kasir-B");
    Thread kasir3 = new Thread(task, "Kasir-C");

    kasir1.start();
    kasir2.start();
    kasir3.start();

    kasir1.join();
    kasir2.join();
    kasir3.join();

    System.out.println("--- HASIL AKHIR STOK: " + ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
  }
}
```

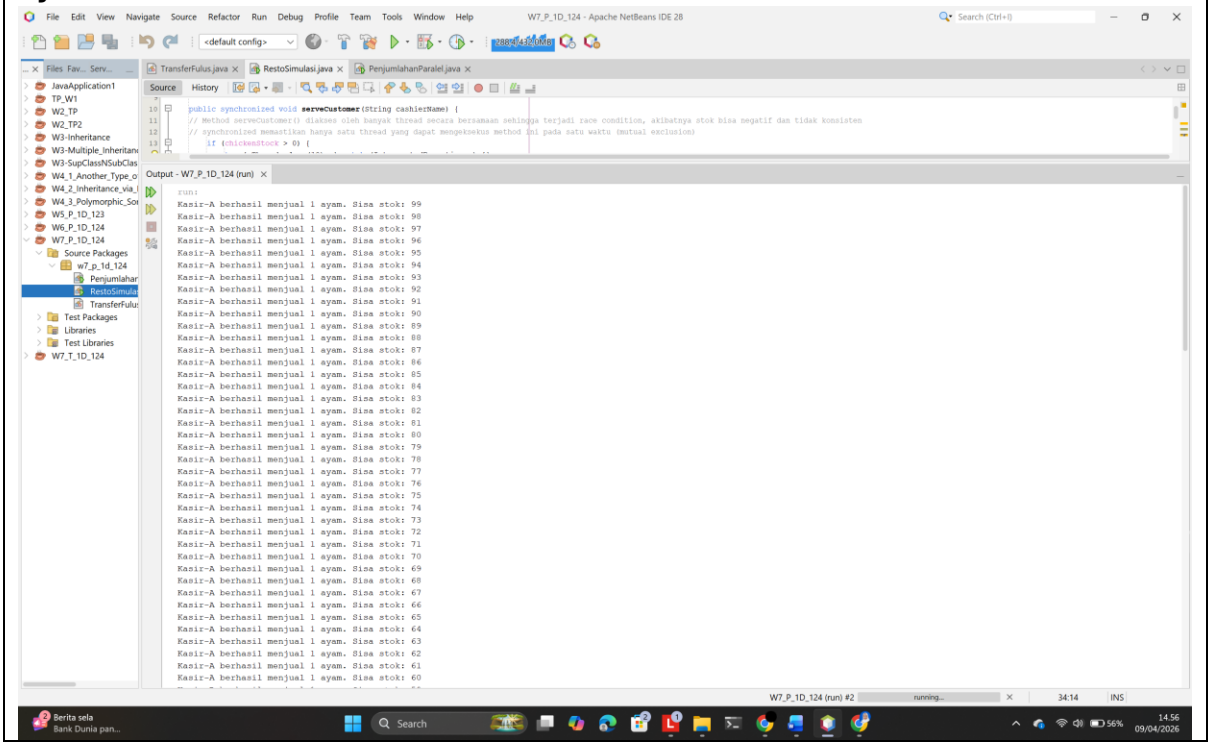
- **ReentrantLock**

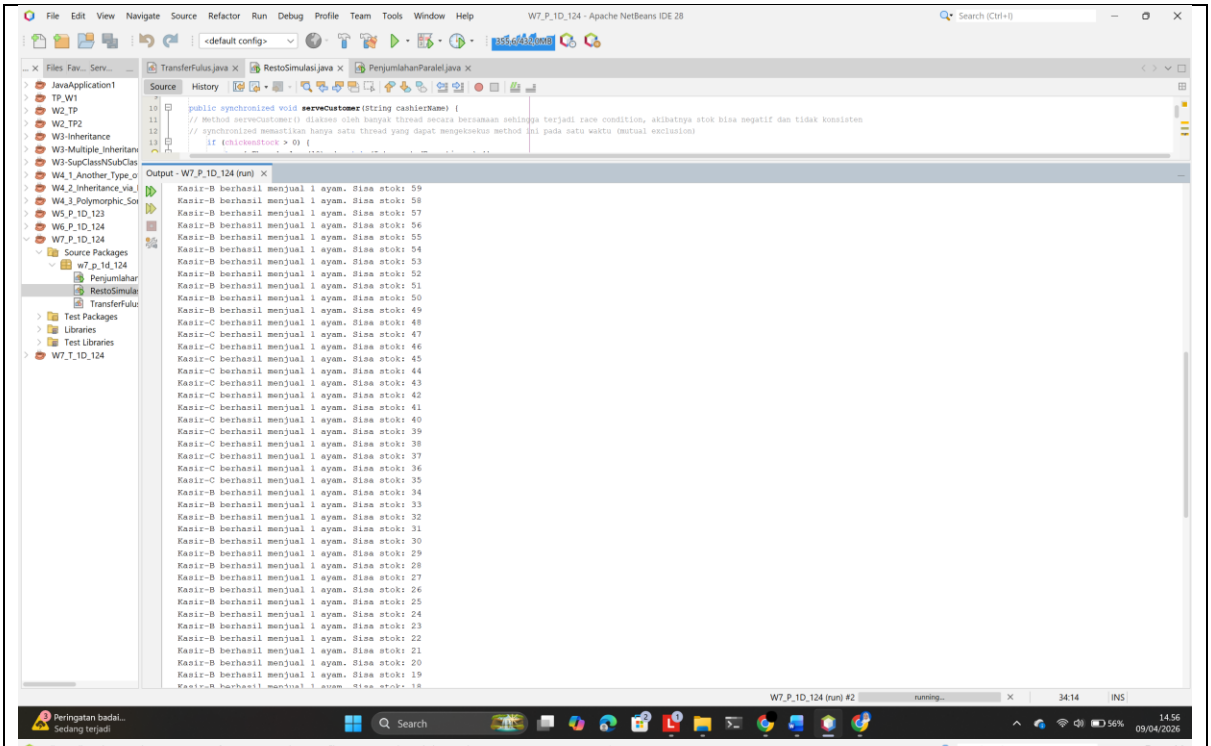
```
package w7_p_1d_124;

/**
 * @author ZAHRA
 */
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class Resto {
  private int chickenStock = 100;

  private final Lock lock = new ReentrantLock(); //Membuat objek kunci (lock)

  public void serveCustomer(String cashierName) {
    // Mengunci area
    lock.lock();
    try {
      if (chickenStock > 0) {
        try { Thread.sleep(10); } catch (InterruptedException e) {}
        chickenStock--;
        System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " + chickenStock);
      } else {
        System.out.println(cashierName + " gagal: Stok Habis!");
      }
    } finally {
      lock.unlock(); //Melepaskan kunci agar tidak terjadi deadlock
    }
  }

  public int getRemainingStock() {
    return chickenStock;
  }
}

public class RestoSimulasi_RL {
  public static void main(String[] args) throws InterruptedException {
    Resto ayamJuicyLuicyGallagher = new Resto();
```

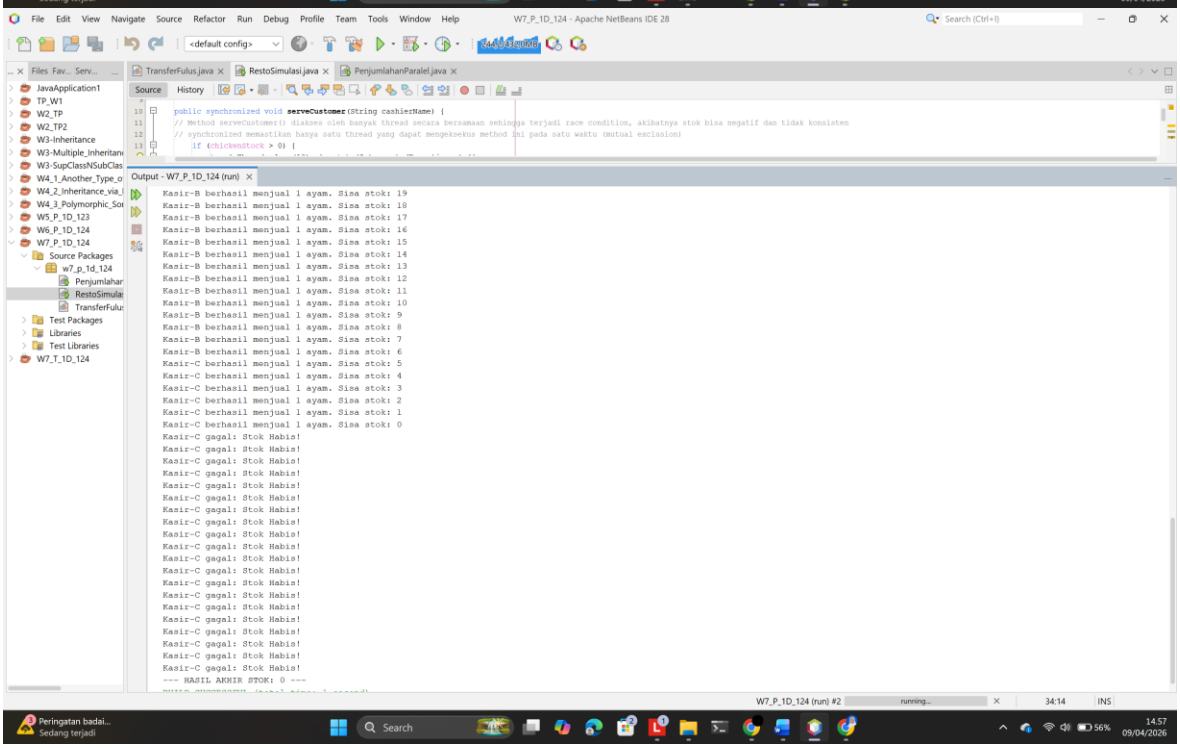
```
    Runnable task = () -> {
      for (int i = 0; i < 40; i++) {
        ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
      }
    };

    Thread kasir1 = new Thread(task, "Kasir-A");
    Thread kasir2 = new Thread(task, "Kasir-B");
    Thread kasir3 = new Thread(task, "Kasir-C");

    kasir1.start();
    kasir2.start();
    kasir3.start();

    kasir1.join();
    kasir2.join();
    kasir3.join();

    System.out.println("--- HASIL AKHIR STOK: " + ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
  }
}
```

Screenshot Hasil

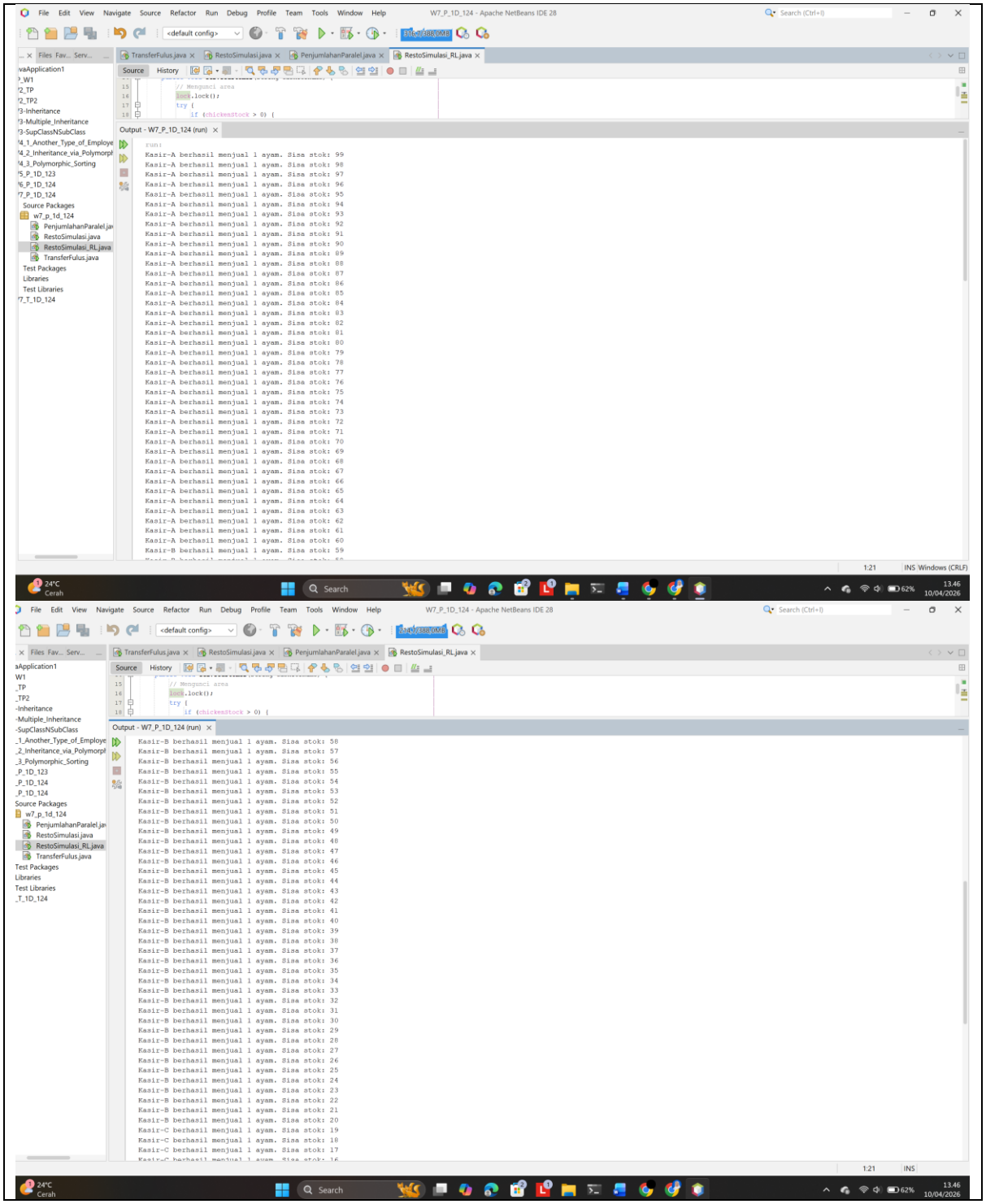
Synchronized



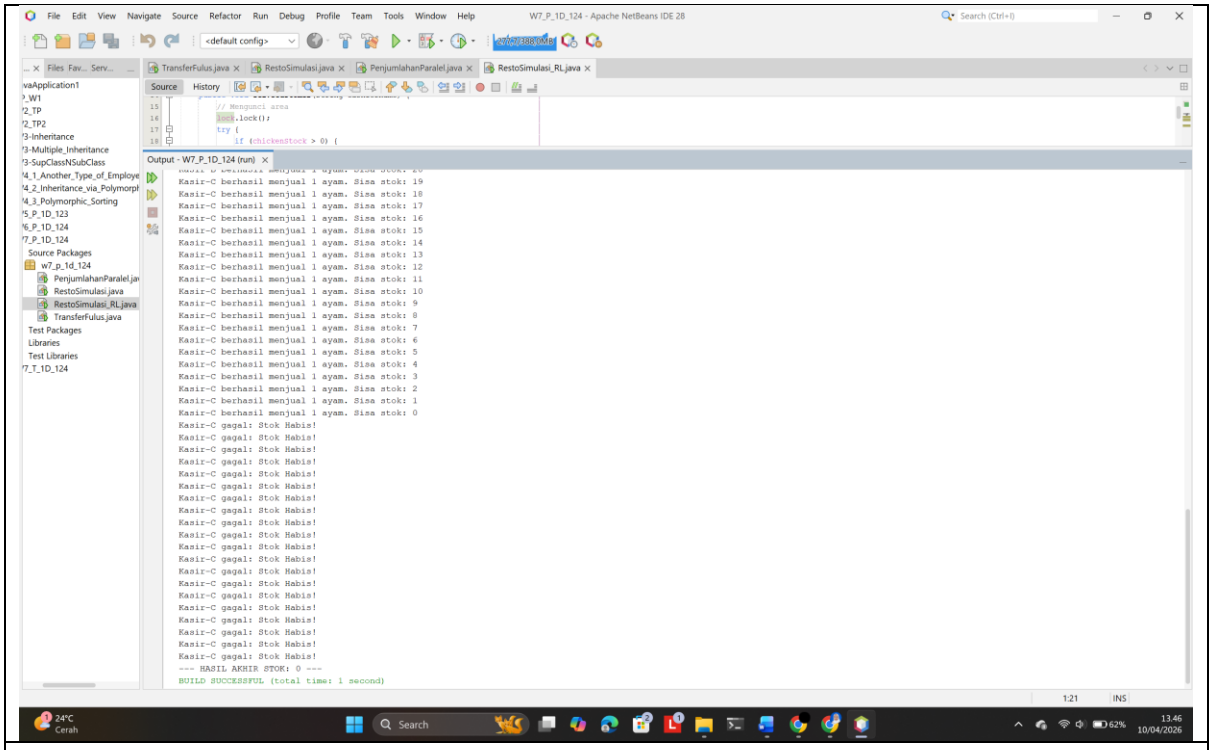




ReentrantLock


W7_P_1D_124 - Apache NetBeans IDE 28
TransferFulus.java
RestoSimulasi.java
PenjumlahanParalel.java
RestoSimulasi_RL.java
// Mengunci area
lock.lock();
try {
if (chickenStock > 0) {
Output - W7_P_1D_124 (run)
run:
Kasir-A berhasil menjual 1 ayam. Sisa stok: 99
Kasir-A berhasil menjual 1 ayam. Sisa stok: 98
Kasir-A berhasil menjual 1 ayam. Sisa stok: 97
Kasir-A berhasil menjual 1 ayam. Sisa stok: 96
Kasir-A berhasil menjual 1 ayam. Sisa stok: 95
Kasir-A berhasil menjual 1 ayam. Sisa stok: 94
Kasir-A berhasil menjual 1 ayam. Sisa stok: 93
Kasir-A berhasil menjual 1 ayam. Sisa stok: 92
Kasir-A berhasil menjual 1 ayam. Sisa stok: 91
Kasir-A berhasil menjual 1 ayam. Sisa stok: 90
Kasir-A berhasil menjual 1 ayam. Sisa stok: 89
Kasir-A berhasil menjual 1 ayam. Sisa stok: 88
Kasir-A berhasil menjual 1 ayam. Sisa stok: 87
Kasir-A berhasil menjual 1 ayam. Sisa stok: 86
Kasir-A berhasil menjual 1 ayam. Sisa stok: 85
Kasir-A berhasil menjual 1 ayam. Sisa stok: 84
Kasir-A berhasil menjual 1 ayam. Sisa stok: 83
Kasir-A berhasil menjual 1 ayam. Sisa stok: 82
Kasir-A berhasil menjual 1 ayam. Sisa stok: 81
Kasir-A berhasil menjual 1 ayam. Sisa stok: 80
Kasir-A berhasil menjual 1 ayam. Sisa stok: 79
Kasir-A berhasil menjual 1 ayam. Sisa stok: 78
Kasir-A berhasil menjual 1 ayam. Sisa stok: 77
Kasir-A berhasil menjual 1 ayam. Sisa stok: 76
Kasir-A berhasil menjual 1 ayam. Sisa stok: 75
Kasir-A berhasil menjual 1 ayam. Sisa stok: 74
Kasir-A berhasil menjual 1 ayam. Sisa stok: 73
Kasir-A berhasil menjual 1 ayam. Sisa stok: 72
Kasir-A berhasil menjual 1 ayam. Sisa stok: 71
Kasir-A berhasil menjual 1 ayam. Sisa stok: 70
Kasir-A berhasil menjual 1 ayam. Sisa stok: 69
Kasir-A berhasil menjual 1 ayam. Sisa stok: 68
Kasir-A berhasil menjual 1 ayam. Sisa stok: 67
Kasir-A berhasil menjual 1 ayam. Sisa stok: 66
Kasir-A berhasil menjual 1 ayam. Sisa stok: 65
Kasir-A berhasil menjual 1 ayam. Sisa stok: 64
Kasir-A berhasil menjual 1 ayam. Sisa stok: 63
Kasir-A berhasil menjual 1 ayam. Sisa stok: 62
Kasir-A berhasil menjual 1 ayam. Sisa stok: 61
Kasir-A berhasil menjual 1 ayam. Sisa stok: 60
Kasir-B berhasil menjual 1 ayam. Sisa stok: 59

W7_P_1D_124 - Apache NetBeans IDE 28
TransferFulus.java
RestoSimulasi.java
PenjumlahanParalel.java
RestoSimulasi_RL.java
// Mengunci area
lock.lock();
try {
if (chickenStock > 0) {
Output - W7_P_1D_124 (run)
Kasir-B berhasil menjual 1 ayam. Sisa stok: 58
Kasir-B berhasil menjual 1 ayam. Sisa stok: 57
Kasir-B berhasil menjual 1 ayam. Sisa stok: 56
Kasir-B berhasil menjual 1 ayam. Sisa stok: 55
Kasir-B berhasil menjual 1 ayam. Sisa stok: 54
Kasir-B berhasil menjual 1 ayam. Sisa stok: 53
Kasir-B berhasil menjual 1 ayam. Sisa stok: 52
Kasir-B berhasil menjual 1 ayam. Sisa stok: 51
Kasir-B berhasil menjual 1 ayam. Sisa stok: 50
Kasir-B berhasil menjual 1 ayam. Sisa stok: 49
Kasir-B berhasil menjual 1 ayam. Sisa stok: 48
Kasir-B berhasil menjual 1 ayam. Sisa stok: 47
Kasir-B berhasil menjual 1 ayam. Sisa stok: 46
Kasir-B berhasil menjual 1 ayam. Sisa stok: 45
Kasir-B berhasil menjual 1 ayam. Sisa stok: 44
Kasir-B berhasil menjual 1 ayam. Sisa stok: 43
Kasir-B berhasil menjual 1 ayam. Sisa stok: 42
Kasir-B berhasil menjual 1 ayam. Sisa stok: 41
Kasir-B berhasil menjual 1 ayam. Sisa stok: 40
Kasir-B berhasil menjual 1 ayam. Sisa stok: 39
Kasir-B berhasil menjual 1 ayam. Sisa stok: 38
Kasir-B berhasil menjual 1 ayam. Sisa stok: 37
Kasir-B berhasil menjual 1 ayam. Sisa stok: 36
Kasir-B berhasil menjual 1 ayam. Sisa stok: 35
Kasir-B berhasil menjual 1 ayam. Sisa stok: 34
Kasir-B berhasil menjual 1 ayam. Sisa stok: 33
Kasir-B berhasil menjual 1 ayam. Sisa stok: 32
Kasir-B berhasil menjual 1 ayam. Sisa stok: 31
Kasir-B berhasil menjual 1 ayam. Sisa stok: 30
Kasir-B berhasil menjual 1 ayam. Sisa stok: 29
Kasir-B berhasil menjual 1 ayam. Sisa stok: 28
Kasir-B berhasil menjual 1 ayam. Sisa stok: 27
Kasir-B berhasil menjual 1 ayam. Sisa stok: 26
Kasir-B berhasil menjual 1 ayam. Sisa stok: 25
Kasir-B berhasil menjual 1 ayam. Sisa stok: 24
Kasir-B berhasil menjual 1 ayam. Sisa stok: 23
Kasir-B berhasil menjual 1 ayam. Sisa stok: 22
Kasir-B berhasil menjual 1 ayam. Sisa stok: 21
Kasir-B berhasil menjual 1 ayam. Sisa stok: 20
Kasir-C berhasil menjual 1 ayam. Sisa stok: 19
Kasir-C berhasil menjual 1 ayam. Sisa stok: 18
Kasir-C berhasil menjual 1 ayam. Sisa stok: 17



Penjelasan

Pada soal ini, masalah yang terjadi adalah race condition, di mana terjadi dua atau lebih thread mengakses dan memodifikasi data yang sama secara bersamaaan, tanpa mekanisme pangamanan, sehingga hasil akhir menjadi tidak konsisten/error. Kondisi ini muncul karena terdapat sebuah shared resource (variabel chickenstock) yang diakses dan dimodifikasi oleh lebih dari satu thread (Kasir A, Kasir B, dan Kasir C) secara bersamaan tanpa pengamanan. Akibatnya, terjadi tumpang tindih data (lost update), di mana beberapa kasir mencetak sisa stok yang sama atau stok menjadi minus.

- Synchronized

Solusi untuk mengatasi race condition dalam kasus ini adalah dengan menerapkan synchronization, yang dilakukan dengan menambahkan keyword synchronized pada deklarasi method serveCustomer. Penambahan keyword ini menerapkan Mutual Exclusion, yang memastikan bahwa method tersebut hanya boleh dimasuki dan dieksekusi satu thread pada satu waktu. Ketika satu kasir memproses pengurangan stok di dalam method, kasir lainnya akan diblokir sementara dengan menunggu hingga kasir pertama selesai. Dengan mekanisme ini, proses manipulasi data stok ayam menjadi aman, berurutan, dan terhindar tumpang tindih data.

- ReentrantLock

Salah satu solusi lain yang bisa dilakukan adalah dengan menggunakan class ReentrantLock. Berbeda dengan synchronized yang mengunci secara implisit (otomatis), ReentrantLock memberikan kontrol penguncian secara eksplisit (manual). Pada kode, thread memanggil lock.lock() sehingga selama area tersebut dikunci, kasir lain yang mencoba masuk akan menunggu/mengantre hingga kasir sebelumnya selesai. Kemudian terdapat pelepasan kunci lock.unlock() yang diletakkan dalam blok

finally yang memastikan apapun yang terjadi dalam proses termasuk jika terjadi error/exception, kunci akan dibuka kembali sehingga sistem terhindar dari deadlock.

Penemuan :

- Pemilihan lock.lock() dibanding trylock()
  Karena trylock() bersifat non-blocking sehingga jika kasir mendapati pintu terkunci akan langsung membatalkan penjualan yang mengakibatkan banyak transaksi terbuang dan stok 100 ayam tidak akan pernah habis.
- Pemilihan private final Lock lock = new ReentrantLock(); dibanding private final ReentrantLock lock = new ReentrantLock();
  Karena dengan mendeklarasikan variabel sebagai tipe Lock, kode menjadi loosely coupled pada satu jenis gembok tertentu. Lock merupakan interface yang mendefinisikan fungsi dasar menjadi objek sinkronisasi, sedangkan ReentrantLock adalah concrete class dari interface tersebut. Daripada itu kelas ReentrantLock memiliki berbagai fungsi lain yang tidak digunakan dalam interface Lock. Dengan membatasi tipe referensi menjadi Lock, akses terhadap fungsi-fungsi tersebut mejadi tertutup sehingga sistem menggunakan operasi blocking standar yang aman untuk Thread Safety.

Berdasarkan implementasi yang dilakukan, permasalahan Race Condition pada share resource berhasil diselesaikan dengan prinsip Mutual Exclusion dengan dua metode yang diuji, yaitu synchronized dan ReentrantLock. Keduanya terbukti efektif menahan akses konkuren dari berbagai thread, sehingga manipulasi data berjalan sekuensial dan terhindar dari lost update. Pendekatan synchronized menawarkan solusi ringkas dengan penguncian implisit (otomatis), sedangkan RentrantLock memberikan kontrol penguncian eksplisit (manual) dengan keamanan ekstra seperti penggunaan blok finally yang dilengkapi unlock untuk mencegah Deadlock.
Dalam kasus soal ini, penggunaan synchronized lebih baik dibanding ReentrantLock karena untuk mencegah overengineering, method serveCustomer hanya mengurangi angka yang sederhana, sehingga menggunakan ReentrantLock menciptakan objek penguncian yang kompleks, padahal dalam kasus ini fitur-fitur spesial ReentrantLock tidak digunakan.

Nama Teman Hal yang Dibantu (Opsional)

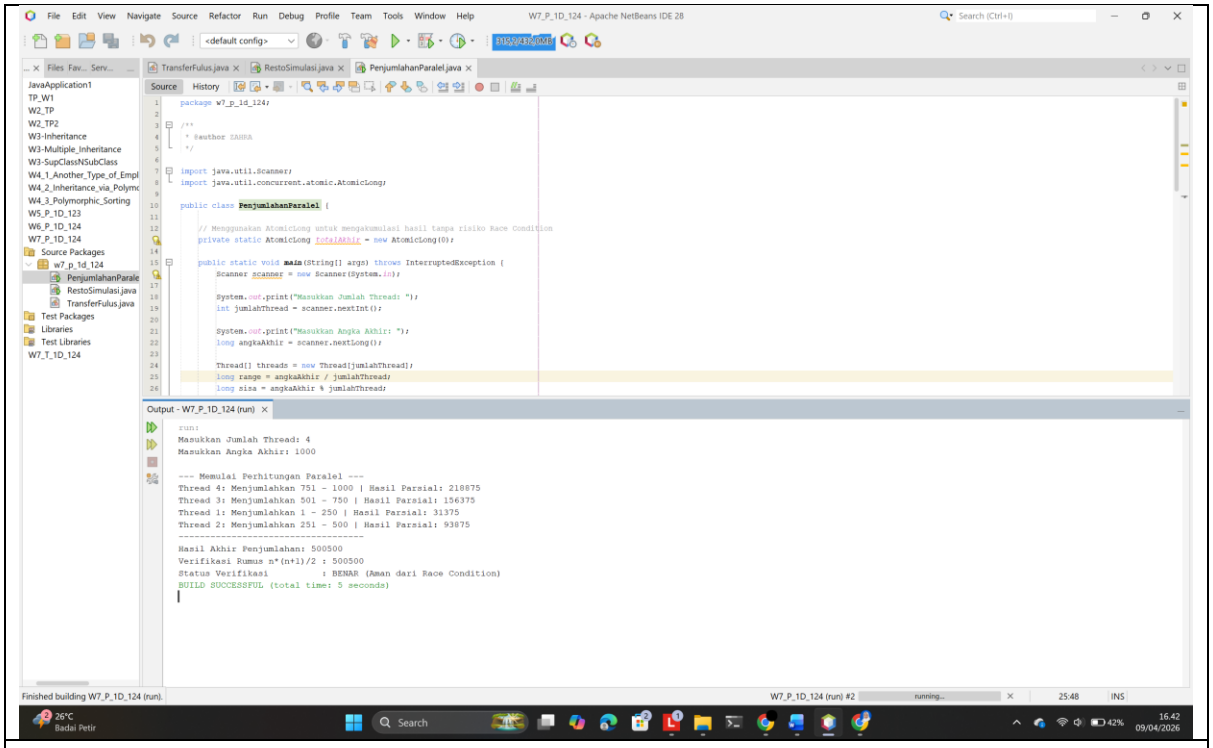
No. 3 Penjumlahan Paralel

Soal Praktikum
Instruksi Pengerjaan: Buatlah suatu program yang bertujuan untuk melakukan penjumlahan secara paralel dengan ketentuan sebagai berikut: - Menerima Input: (1) Jumlah Thread; (2) Angka Akhir - Contoh: Menjumlahkan angka dari 1 sampai "Angka Akhir". Jika "Jumlah Thread" = 4, "Angka Akhir" = 1000, maka: - Thread 1: Menjumlahkan 1 - 250 - Thread 2: Menjumlahkan 251 - 500 - Thread 3: Menjumlahkan 501 - 750 - Thread 4: Menjumlahkan 751 - 1000 - Hasil akhir diperoleh dari hasil parsial (dari masing-masing thread) yang dijumlahkan menjadi total akhir. - Perhatikan Thread Safety, Mekanisme Pembagian Tugas (Divide and Conquer), dan Syncronization agar hasil akhir sesuai! - Pada program, tampilkan output: - Tugas apa yang dikerjakan oleh masing-masing thread - Hasil parsial dari masing-masing thread - Hasil akhir yang merupakan penjumlahan dari semua hasil parsial - Berikan penjelasan yang lengkap untuk setiap bagian pentingnya!
Source Code

```
package w7_p_1d_124;

/**
 * @author ZAHRA
 */

import java.util.Scanner;
import java.util.concurrent.atomic.AtomicLong;

public class PenjumlahanParalel {

    // Menggunakan AtomicLong untuk mengakumulasi hasil tanpa risiko Race Condition
    private static AtomicLong totalAkhir = new AtomicLong(0);

    public static void main(String[] args) throws InterruptedException {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Masukkan Jumlah Thread: ");
        int jumlahThread = scanner.nextInt();

        System.out.print("Masukkan Angka Akhir: ");
        long angkaAkhir = scanner.nextLong();

        Thread[] threads = new Thread[jumlahThread];
        long range = angkaAkhir / jumlahThread;
        long sisa = angkaAkhir % jumlahThread;

        long startAngka = 1;
```

```
    System.out.println("\n--- Memulai Perhitungan Paralel ---");

    for (int i = 0; i < jumlahThread; i++) {
      final int idThread = i + 1;
      final long start = startAngka;

      // Thread terakhir menangani sisa pembagian (jika ada)
      final long end = (i == jumlahThread - 1) ? (startAngka + range - 1 + sisa) : (startAngka + range - 1);

      threads[i] = new Thread(() -> {
        long hasilParsial = 0;

        // Menjumlahkan rentang angka pada thread masing-masing (Divide and Conquer)
        for (long j = start; j <= end; j++) {
          hasilParsial += j;
        }

        System.out.println("Thread " + idThread + ": Menjumlahkan " + start + " - " + end + " | Hasil
Parsial: " + hasilParsial);

        // Sinkronisasi secara atomic untuk memperbarui totalAkhir
        totalAkhir.addAndGet(hasilParsial);
      });

      startAngka = end + 1;
      threads[i].start();
    }

    // Memastikan main thread menunggu semua thread pekerja selesai
    for (int i = 0; i < jumlahThread; i++) {
      threads[i].join();
    }

    System.out.println("-----------------------------------");
    System.out.println("Hasil Akhir Penjumlahan: " + totalAkhir.get());

    // Membuktikan hasil thread dengan rumus deret aritmatika n * (n + 1) / 2
    long expected = angkaAkhir * (angkaAkhir + 1) / 2;
    System.out.println("Verifikasi Rumus n*(n+1)/2 : " + expected);
    System.out.println("Status Verifikasi        : " + (totalAkhir.get() == expected ? "BENAR (Aman dari
Race Condition)" : "SALAH (Terjadi Race Condition)"));

    scanner.close();
  }
}
```

Screenshot Hasil



Penjelasan

Soal ini meminta untuk membuat suatu program, di mana berdasar ketentuan yang ada tantangan dalam pengerjaan program adalah melakukan perhitungan matematis secara paralel dengan membagi beban kerja secara adil kepada setiap thread dan menggabungkan hasil perhitungan dengan aman. Jika beberapa thread mencoba menambahkan hasil perhitungan sementara ke dalam satu variabel shared resource (totalAkhir) secara bersamaan tanpa mekanisme pengamanan, akan terjadi race condition yang menyebabkan terjadinya lost update sehingga hasil penjumlahan menjadi tidak akurat.

Solusi pada program ini menerapkan konsep multithreading:

- Divide and conquer
  Program membagi total rentang angka dengan jumlah Thread di mana setiap batas awal dan akhir secara spesifik menjadi hasilParsial.
- Thread safety
  Untuk mengatasi race condition saat penggabungan data, variabel totalAkhir menggunakan tipe data AtomicLong. Pemanggilan method .addAndGet(hasilParsial) menjamin operasi penambahan bersifat atomic (satu kesatuan yang tidak bisa disela thread lain).
- Mekanisme join
  Sama seperti pada soal-soal sebelumnya, dalam soal ini digunakan pula method.join() pada looping eksekusi untuk menahan main thread menunggu semua thread menyelesaikan tugasnya sebelum mencetak hasil akhir.
- Verifikasi data
  Untuk membuktikan keakuratan sistem multithreading, program menambhakan sistem validasi menggunakan rumus deret aritmatika (n x (n+1)/2). Jika hasil akhir thread persis dengan hasil rumus, terbukti bahwa

program berjalan dengan aman dan sesuai tanpa ada data yang hilang akibat race condition.
Nama Teman Hal yang Dibantu (Opsional)